

A. A Constant-Depth Algorithm Exists for S_3

S_5 is the smallest non-solvable permutation group. S_3 is isomorphic to D_3 , the symmetry group of an equilateral triangle, which can be generated by a transposition $a = (23)$ and a 3-cycle $b = (123)$.⁶ These generators satisfy the relation $ab = ba^{-1}$, which allows any word problem in S_3 to be reduced by tracking (1) the cumulative parity of transpositions and (2) the count of 3-cycles modulo 3. Since both parity checking and modular counting can be computed using constant-depth threshold circuits, the word problem for S_3 belongs to TC^0 .

B. Full Activation Patching Results

In the activation patching experiments, we overwrite (“patch”) portions of the LM’s internal representations and compute how much the resulting logits have changed. As discussed in Section 2.3, we perform prefix patching to localize important token positions. However, in addition to prefix patching, we also explore the following types of localization methods:

1. In *suffix patching*, all tokens starting from t up to *one before* the last token of the sequence $(h_{t:|A|-1,l})$ are patched at a particular layer l .⁷
2. In *window patching*, all tokens in a w -width window starting from t $(h_{t:t+w-1,l})$ are patched at layer l .

For each of the above localization techniques, we patch the representation with several different *types of content*:

1. In *representation deletion*, we overwrite target representation(s) entirely with a zero vector,

$$h_{t,l} = \mathbf{0}$$

and measure the NLD as follows:

$$\text{NLD} = \frac{\text{LD}(a_1 \dots a_t) - \text{LD}(a_1 \dots a_t; h_{t,l} \leftarrow \mathbf{0})}{\text{LD}(a_1 \dots a_t)}$$

2. In *representation substitution*, we overwrite the representation(s) with those derived from running the LM on a minimally different (corrupted) representation P_{corrupt} . This is the setting described in Section 2.3.

Full results are shown in Figure 8. In general, we discover the following:

In PAA models, parities are computed in parallel in early-mid layers We use prefix substitution patching described in Section 2.3, but plot pairs that have *same parity* final states ($\epsilon(\hat{y}) = \epsilon(\hat{y}')$) separately from pairs that have *opposite parity* final states ($\epsilon(\hat{y}) \neq \epsilon(\hat{y}')$).

Results are shown in Figure 8A (for same parity) and 8B (for opposite parity). We find that in AA models, parities are computed with the state – with both the same-parity and opposite-parity patching patterns displaying the same exponential curve. However, in PAA models, the patching patterns for same- and opposite-parities differ drastically. When parities are the same, only the parity complement must be computed to infer the final state; the patching pattern in this case indicates that the parity complement is computed in an associative manner. When the parities are different, the patching signature has a component that resembles a parallel patching signature, which is where the parity is computed. Restoring prefixes of layers before the parity is computed results in the entire prediction being of the correct parity, while restoring prefixes of layers after that results in the entire prediction being of the incorrect parity. We see that parities are computed roughly in parallel at early layers (around layers 3-5). Note that there is a middle region where restoring the prefixes shifts the logits towards the correct prediction, but not 100%: when prefixes in these regions are restored, the LM does not know the parity of the final answer, but does know some aspects of the parity complement, which was computed in an associative manner.

⁶https://proofwiki.org/wiki/Symmetric_Group_on_3_Letters_is_Isomorphic_to_Dihedral_Group_D3

⁷We do not patch the last token as the last token residual contains the current accumulated information necessary for computing the final product, and almost always will destroy the prediction when patched, making this method uninformative as localization tool. We wish to see what *other* tokens the final token uses when constructing its representation.

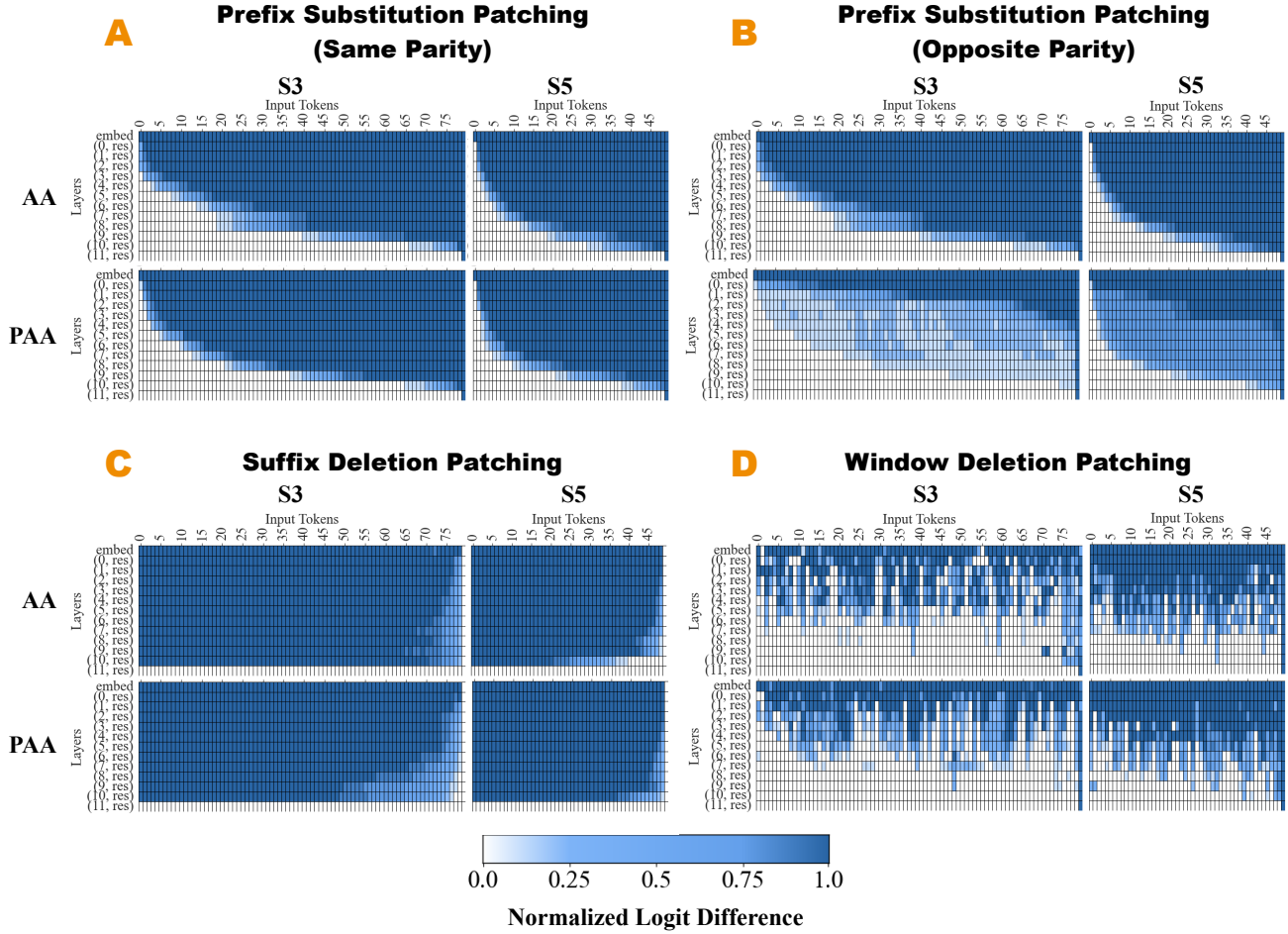


Figure 8. Activation patching results across different types of localization (prefix, suffix, window patching) and different types of patching content (substitution, deletion). (A): Prefix-substitution results on only sequences with the *same* parity. We see the same exponential patching pattern in both AA and PAA models, showing that parity complements are computed in the same associative manner. (B): Prefix-substitution results on only sequences with *opposite* parities. We see the exponential patching pattern in AA models, meaning that in AA models, parities are computed with the state. In PAA models, however, the patching pattern is roughly parallel, meaning parities are computed roughly in parallel in early layers. (C): Suffix-deletion results show that we can ignore progressively longer sequences of suffixes as we go down the layers of the network, consistent with how we believe AA and PAA work. (D): Window-deletion results show that important activations are arranged hierarchically, again consistent with how we believe AA and PAA work.

Increasingly longer suffixes are ignored for AA and PAA models in later layers In Figure 8C, we show suffix deletion patching results, finding that we can swap out *exponentially longer* both AA and PAA models without affecting the prediction. This is in line with how the associative algorithm in either model works: suffixes of progressively longer lengths are collected into the final token as we go down the layers.

Important activations are arranged hierarchically In Figure 8D, we show window deletion patching results, with a window size of 1. We find a patching pattern consistent with the associative algorithm: deleting any single token in the early layers is extremely important, but the spacing of important tokens gets sparser as we go down the layers, consistent with the depiction of AA/PAA in Figure 1. At the bottom layers, deleting any single token is unimportant for the final computation of the state.

Patching signatures are relatively consistent across examples In Figure 9, we plot the standard deviations across 200 pairs of S3 inputs for the following three sets of results: (A) prefix substitution patching of AA models, (B) prefix

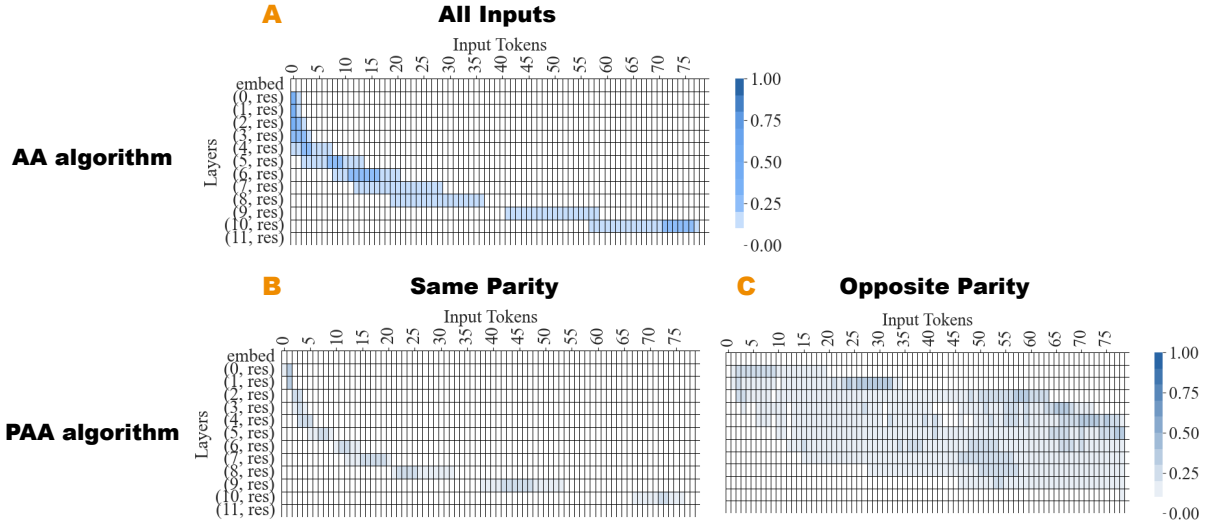


Figure 9. Standard deviations of the prefix substitution activation patching results across 200 S_3 input pairs, for (A) AA models, (B) PAA models on pairs with the same parity, and (C) PAA models on pairs with opposite parity. We find generally low standard deviations across examples.

substitution patching of PAA models (opposite parity inputs), (C) prefix substitution patching of PAA models (same parity inputs). We find relatively low standard deviations in all three cases, showing that these signatures hold across different examples.

C. Full Probing Results

C.1. Probing signatures are relatively consistent across examples

We investigate the sensitivity of our probing signatures to the data on which the probe was trained. We focus on Pythia models trained on S_3 . For each model type (PAA vs. AA) and each probe type (parity vs. state probe), we train 10 different probes across 10 different random subsets of the S_3 dataset, and report the standard deviations of their accuracies across the 10 runs. Results can be found in Table 1. We find low standard deviations (less than 10^{-3}) in all four cases, indicating that our probe signatures are robust to different subsets of the data and to randomness in probe training.

C.2. Probe accuracies over sequence lengths

How do the probe accuracies in Figure 3 decompose over sequence lengths? We sweep over S_3 sequences $a_1 \dots a_i$ of lengths ranging from $i = 5$ to 100, and train a linear probe that takes in input $h_{t,l}|a_1 \dots a_t$ —the layer- l , position- t representation of the model on input sequence $a_1 \dots a_i$ with $t < i$ — and aims to predict the final state s_i from the hidden representation.

The mean probability the probe put on the correct answer is plotted in Figure 10. We find that, generally speaking, both AA and PAA linearly encode states of exponentially longer sequences as they go down the layers. We find evidence that the PAA models use their intermediate layers to compute parity in parallel: at around the second residual layer, PAA models place $\frac{1}{3}$ probability on the correct answer (there are three actions of each parity in S_3).

C.3. Examples of Associative Algorithm Representations that Do or Do Not Linearly Encode Parity

As shown in Figure 1, models that learn AA sometimes encode parity linearly at the final layer but sometimes do not. The examples shown in Figure 3 all do *not* linearly encode parity at the final layer. We show a 3D visualization of the S_3 AA model’s final hidden representations along the three principal components of the representation (which explain 41.8% of the variance in the data) in Figure 11. As we can see, parity is *not* linearly encoded at the final layer. In Figure 12, we show